A numerical refinement of the aligned-cache APSP exponent

20 October 2026

Abstract

We give exact rational parameters satisfying all 47 numerical conditions used by the current leading claimed APSP construction. They yield the conditional bound $O(n^{2.9929479187094})$, improving the displayed $O(n^{2.99294792})$ bound by 1.2906×10^{-9} in its exponent. This is a parameter and numerical-certificate refinement of the existing algorithm. Its APSP consequence depends on that construction's written algebraic and runtime proof; it is not a new Lean verification. A standalone Python checker using rational interval arithmetic accompanies this note. The numerical feasibility result is independent of floating-point optimization.

1 Scope and precise status

The problem is deterministic all-pairs shortest paths in a directed graph with n vertices, polynomially bounded integer weights, and no negative cycles. The challenge requires reachability and distance outputs, with edge-presence and weight matrices as inputs. For each fixed input-magnitude exponent, its word-RAM program must work at every sufficiently large word width proportional to $\log n$ [1].

The baseline is the aligned Walsh embedding, polynomial query cache, and composite-incidence reduction in the note by the GitHub author `shreshth-s` [2], building on Alman–Vassilevska Williams [3] and Majdoddin [4]. The pinned source revision is `8a21eaae8159`; its full commit identifier is recorded in `parameters.json`. The challenge classifies its exponent 2.99294792 as *Claimed*: the accompanying Lean development uses a different cost model from the required `EndStatement` model [1].

This note proves the numerical feasibility of a new rational witness. Its algorithmic corollary uses the cited written lemmas as dependencies. A separate AI-agent review during preparation checked the local identity, Walsh normalization, cache counting, box decomposition, phase costs, certification rule, and reductions to min-plus and APSP; no defect was found in those examined steps. That review and finite checks do not constitute a formal proof of the complete word-RAM program.

2 Cost expressions inherited from the construction

Write $\mathcal{H}(x) = -x\ln x - (1-x)\ln(1-x)$ and define

$$C_K = \frac{c\mathcal{H}(1/c)}{\ln 4}, \qquad T = c - 1 + C_K, \tag{1}$$

$$G = \frac{c\mathcal{H}(1/c) - c\mathcal{H}((1-\theta)/c) - \theta\ln 4}{\ln 2}, \tag{2}$$

$$q_1 = \frac{\mathcal{H}(\theta)}{\ln 2}, \qquad q_2 = q_1 + 2\theta, \qquad E = T + q_1. \tag{3}$$

The underlying selected-entry data structure has preprocessing $\widetilde{O}(qND^E + qN^2D^{-G} + qND)$ and a specified-coefficient query cost $\widetilde{O}(qD^{q_1} + D^{q_2})$, where q is the cyclic ring length. The layout condition uses T, whereas cache formation uses E.

Let $R > S \geq 1$ be fixed integers, and put

$$\eta = \frac{b}{R}, \qquad u = S\eta, \qquad v = (R-S)\eta = b - u, \qquad e = b + u.$$

Here n^b is the middle-block scale, n^η is the candidate-prime scale up to logarithmic factors, u is the frozen folded-modulus exponent, and v is the final cyclic-modulus exponent.

At the final cyclic endpoint, the four active runtime savings below exponent three are

$$\boxed{eG - \eta, \quad u - eq_1 - \eta, \quad b - eq_2 - \eta, \quad 1 + \frac{u}{2} - eE - \eta.} \tag{4}$$

The matrix-formation and incidence savings are respectively $1 - b - u/2 - \eta$ and $b - \eta$. The layout margin is $1 + u/2 - eT$. The supplied checker also retains the upstream's ordinary-box-encoder checks, even though the aligned cache includes the needed constant coefficients.

3 How the new witness was found

First disregard the arbitrarily small fixed-stage overhead η and balance the four expressions in (4). Equating the first three gives

$$u = e(q_1 + G), \qquad b = e(q_2 + G), \qquad q_1 + q_2 + 2G = 1.$$

The last equality follows from $b + u = e$ and is equivalent to

$$G + q_1 + \theta = \frac{1}{2}. \tag{5}$$

Equating the cache saving gives

$$e = \frac{1}{T + (q_1 + G)/2} = \frac{1}{T + 1/4 - \theta/2}, \qquad \delta_\infty = \frac{G}{T + (q_1 + G)/2}.$$

Thus the search can solve (5) for θ at each c and optimize one scalar objective. A high-precision stationary-point calculation gives approximately

$$\begin{aligned} c_* &= 10.665500244065728984 99, \\ \theta_* &= 0.065297823698726309 90, \\ \delta_* &= 0.007052081290690653281665. \end{aligned}$$

These values guided the rational witness. They do not prove a global optimum or rule out a better algorithm. The certificate below does not depend on the optimizer or on the stationarity equations being solved exactly.

4 The exact rational certificate

Proposition 1. *Take the following decimals as exact rational numbers:*

$$c = 10.665500244065729, \qquad \theta = 0.06529782369872631,$$
$$b = 0.046046352103722105, \qquad \delta = 0.0070520812906,$$
$$R = 10^{12}, \qquad S = 768979037380.$$

With (1)–(3) *and the exact stage identities above, all 47 numerical inequalities checked in the upstream aligned-cache certificate hold strictly for target saving* δ*.*

Proof. The accompanying `verify_certificate.py` evaluates each expression using rational interval arithmetic. Its inputs come from `parameters.json`. Natural logarithms are enclosed by the explicit series-and-remainder argument in Section 5. The following entries are conservative lower bounds, rounded downward from the computed rational interval endpoints:

Final cost component	Certified saving lower bound
Boxes	0.007052081290641657832006957
Residual coefficient queries	0.007052081290622402059199658 9
Stored-box coefficient reads	0.007052081290660913478717303 1
Polynomial cache	0.007052081291050220155817077 4
Ordinary box encodings	0.042460760806650238146551659 1
Matrix formation	0.936249308138441467538932635 4

Each exceeds δ. The smallest runtime gap is greater than $2.24020591996589 \times 10^{-14}$. The final layout margin exceeds 0.035408679516008580314544703 3.

For completeness, the checker verifies both endpoints of both phases. In the scalar phase, $x \in [0, u]$, the five principal savings are

$$b - x + (b + x)G - \eta, \qquad b - (b + x)q_1 - \eta,$$
$$b - (b + x)q_2 - \eta, \qquad 1 + b - x/2 - (b + x)E - \eta,$$
$$1 - 3x/2 - \eta.$$

In the cyclic phase, $z \in [0, v]$, they are

$$b - u - z + eG - \eta, \qquad b - z - eq_1 - \eta,$$
$$b - eq_2 - \eta, \qquad 1 + b - u/2 - z - eE - \eta,$$
$$1 - 3u/2 - z - \eta.$$

Each is affine in its phase variable. Endpoint verification therefore covers the full interval. The checker also verifies the geometric-tail conditions, increasing-binomial-tail conditions, positivity conditions, tile margins, and exact integer stage identities. There are 19 initial checks and 28 endpoint checks. Full directed lower and upper bounds are recorded in `certificate.json`. □

Corollary 2 (Conditional algorithmic consequence)**.** *Assuming the selected-entry oracle, composite-incidence reduction, min-plus/APSP transfers, and word-RAM cost interpretation in the cited written construction, the stated deterministic integer APSP problem is solvable in* $O(n^{2.9929479187094})$ *time.*

Proof. All runtime terms are bounded by $n^{3-\delta-\varepsilon}$ times fixed powers of $\log n$, for some fixed $\varepsilon > 0$, by the strict certificate. These logarithmic factors are absorbed in n^{ε}. The fixed number of stages, rounding overheads, bit-scaling calls, and APSP squarings are included in that accounting. Since $3 - \delta = 2.9929479187094$, the asserted exponent follows under the stated algorithmic dependencies. □

The improvement over the displayed baseline is exactly

$$2.99294792 - 2.9929479187094 = 0.0000000012906.$$

This is an extremely small asymptotic refinement. The stage count is a fixed 10^{12}; both it and the tiny positive slack imply enormous hidden constants and asymptotic thresholds. No practical speedup is claimed.

5 Why the checker is exact

For rational $1 \le y \le 2$, put $z = (y-1)/(y+1)$, so $0 \le z \le 1/3$. The identity

$$\ln y = 2\sum_{j=0}^{\infty} \frac{z^{2j+1}}{2j+1}$$

gives the rational enclosure

$$2\sum_{j=0}^{39} \frac{z^{2j+1}}{2j+1} \;\le\; \ln y \;\le\; 2\sum_{j=0}^{39} \frac{z^{2j+1}}{2j+1} + \frac{2z^{81}}{81(1-z^2)}.$$

The upper remainder follows by replacing every omitted denominator by 81 and summing a geometric series. For an arbitrary positive rational x, the checker finds an integer k with $1 \le x/2^k < 2$, then applies $\ln x = \ln(x/2^k) + k \ln 2$, including the correct interval direction when k is negative. All subsequent arithmetic uses Python's `Fraction`; division requires a strictly positive denominator interval. A condition passes only if its lower interval endpoint is strictly greater than its threshold.

Decimal output uses outward rounding and is not used to decide a check. Explicit exceptions, rather than Python assertions, enforce the checks even under optimized execution. The implementation has no third-party dependencies. Running

```
python3 verify_certificate.py --json certificate.json
```

recomputes all 47 inequalities and exact stage identities.

6 Other avenues examined and verification limits

The search also examined removing channel entries from the polynomial cache, partial channel caching, fixed-profile prefix caching, and larger aligned local identities. The versions with complete cost expressions did not improve the leading exponent. In particular, reconstructing a channel from two ordinary forms reduces preprocessing but increases the residual query exponent; optimizing that tradeoff returned the full cache endpoint. These numerical searches are not impossibility results.

The fact that $I - P$ has bilinear rank two suggests a possible improvement to high-box queries, but the necessary extra stored states lose the ordinary box-tail saving in the straightforward construction. Replacing the query's 4^t factor by 2^t without charging those states would not be justified. A larger improvement needs a new lemma that improves one or more of the four costs in (4).

As corroboration of the inherited proof, three upstream finite checkers were independently rerun: the embedding checker passed 352 original matrix outputs; the cache checker passed 34 arbitrary signed-array queries and 20 encoders; the integration checker passed 530 normalized scalar and cyclic coefficients. These tests do not prove the asymptotic construction. The arithmetic certificate proves numerical feasibility only. This repository records the refinement; it does not constitute a submission to, or verification by, the challenge.

References

[1] Big O Speedrun, *APSP challenge*, accessed 20 October 2026. https://bigospeedrun.com/apsp.

[2] GitHub author `shreshth-s`, *Improving the displayed exponents in arXiv:2610.06783v1*, `thin-matrix-results`, `outputs/exponent-improvement.tex`, revision `8a21eaae8159` (full identifier in the linked source). Pinned source. See especially the aligned Walsh embedding and polynomial-cache lemmas, and equations (A1)–(A13).

[3] Josh Alman and Virginia Vassilevska Williams, *Truly Subquadratic 3SUM and Truly Subcubic APSP via Triangles in Sparse Lopsided Graphs*, arXiv:2610.06783, 2026. https://arxiv.org/abs/2610.06783.

[4] Ruhollah Majdoddin, *Thin matrix products up to $D \leq N^{0.172}$ and lossless reductions for 3SUM, Exact Triangle and APSP, with 3SUM over abelian groups and a dichotomy for coordinatewise problems*, arXiv:2610.17420v1, 2026. https://arxiv.org/abs/2610.17420v1.